

Interview Prep: Two-Tower Retrieval System

Your complete prep guide for Google AIM SWE interviews. Covers every question a recruiter, hiring manager, or technical interviewer might ask about this project. Read this end-to-end before any interview.

Table of Contents

- The 30-Second Pitch
 - Why This Project? (Motivation)
 - Architecture Deep Dive
 - Training Pipeline
 - Loss Functions — The Math
 - Inference & Serving
 - Engineering Practices
 - Bugs I Caught
 - Trade-offs & Limitations
 - Scale: How Would Google Deploy This?
 - Math You Must Know Cold
 - Behavioral Questions
 - Curveball Questions
 - Live Demo Script
-

1. The 30-Second Pitch

"I built a production-style two-tower neural retrieval system on MovieLens-1M — the same architecture YouTube uses for video recommendations. Two separate neural networks encode users and items into a shared 128-dim space, trained with contrastive learning. At serving time, item embeddings are pre-computed in a FAISS index, and only the user tower runs per query — total latency is ~0.4 milliseconds. The model achieves Recall@10 of 0.57, which is 4.3x better than a matrix factorization baseline. I also did a controlled comparison of InfoNCE vs BPR loss and found that BPR won by 68% — the opposite of what the YouTube paper recommends — because ML-1M is heavily popularity-skewed."

Practice this until you can say it in 30 seconds without thinking.

2. Why This Project? (Motivation)

Q: Why did you pick a recommendation system?

Answer:

"I wanted to understand how systems like YouTube and Spotify recommend content at scale. Two-tower retrieval is the actual production architecture used by Google, and I wanted to build it end-to-end — not just train a model in a notebook, but understand the full pipeline from raw data to a serving API with sub-millisecond latency."

Q: Why two-tower specifically? Why not a single model?

Answer:

"Two key reasons:

- 1. **Inference scale:** With a single model that takes (user, item) and outputs a score, I'd have to score every item online — $O(N)$ forward passes per request. With two towers, item embeddings are pre-computed once and indexed. Only the user tower runs at query time.*
- 2. **It's what Google actually does** — the YouTube paper (Yi et al. 2019) uses this exact architecture in production."*

Q: Why MovieLens?

Answer:

"It's the standard academic benchmark — papers from RecSys, KDD, and SIGIR all use it. Lets me compare against published baselines. I started with ML-100K to get the pipeline working, then scaled to ML-1M (1 million ratings) for the real numbers."

Q: Why not a bigger dataset like ML-25M or Amazon Reviews?

Answer:

"ML-1M is the sweet spot for my hardware — trains in under 10 minutes on my M-series Mac, but is large enough to surface real ML challenges like popularity skew. Scaling to ML-25M is straightforward — just point the config at a different file. Production at Google would run distributed training with `DistributedDataParallel`."

3. Architecture Deep Dive

Q: Walk me through the architecture.

Answer:

"Two independent neural networks — one for users, one for items.

Each tower has three layers:

1. **Embedding lookup** — maps an integer ID to a 128-dim vector
 2. **MLP** — $\text{Linear}(128 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256 \rightarrow 128)$ — adds non-linearity and capacity
 3. **L2 normalization** — projects vectors onto the unit hypersphere
- Output is a normalized 128-dim vector. The score between a user and an item is just their dot product — which equals cosine similarity because both are unit vectors."*

Q: Why L2 normalize the embeddings?

Answer:

"Three reasons:

1. **Bounds the score** between -1 and 1, so the loss is well-conditioned
2. **Lets us use FAISS IndexFlatIP** (inner product) which is exact and fast — equivalent to cosine similarity but without an extra normalization step at query time
3. **Stabilizes training** — without it, the model can artificially inflate scores by growing embedding magnitudes, which collapses to a degenerate solution"

Q: Why an MLP after the embedding? Why not just use the embedding directly?

Answer:

"I tested this in my ablation study — without the MLP, Recall@10 collapses from 0.27 to 0.09 (a 3x drop). The MLP adds non-linearity that lets the model learn more complex user-item interactions than a pure linear factorization. It's also where the model can learn user-tower-specific transformations that don't have to mirror the item tower."

Q: What's the dimension of the embedding space and why?

Answer:

"128. I tried 64 and 256 in the ablation:

- D=64: Recall@10 = 0.2714 — slightly underfitting
- D=128: Recall@10 = 0.2722 — sweet spot
- D=256: Recall@10 = 0.2963 — marginally better but 2x memory and slower

*The model is **data-bound**, not **capacity-bound** at this scale. Going bigger doesn't help much."*

4. Training Pipeline

Q: Walk me through your training pipeline.

Answer:

"Five stages:

1. **Load** raw interactions from MovieLens (userid, itemid, rating, timestamp)
2. **Filter** to implicit positives (rating ≥ 4) — turning explicit ratings into the implicit feedback setup that real recommenders see
3. **Time-based split** — sort by timestamp, take first 80% as train, last 20% as test
4. **Encode IDs** — map raw IDs to contiguous 0-indexed integers; save mappings for inference
5. **Train** with BPR contrastive loss + popularity-weighted negative sampling for 25 epochs"

Q: Why time-based split instead of random?

Answer:

*"Random splits leak future information into training. If a user has 100 interactions over 6 months, a random 80/20 split lets the model see month-6 interactions while training on month-1 data — that's not how production works. My first attempt used a random split and got Recall@10 of 0.72, which looked too good. Switching to time-based dropped it to 0.47, which is honest. **Lesson: if your numbers look too good, they probably are.**"*

Q: What's negative sampling and why do you need it?

Answer:

*"Recommendation data is implicit — users tell us what they liked but not what they disliked. So we have to manufacture negatives. For each positive (user, item) pair I sample 4 items the user didn't interact with and treat them as negatives. The model learns to score positives higher than negatives. I use **popularity-weighted sampling** with a 0.75 power smoothing — same trick word2vec uses. Popular items the user didn't interact with are more informative negatives because the model is most likely to wrongly recommend them."*

Q: What's the train/test split exactly?

Answer:

"Chronological 80/20. After filtering rating ≥ 4 on ML-1M, I get ~575K positive interactions. 460K go to train, ~115K to test. After encoding, the test set is filtered to only contain users and items seen in training (cold-start cases excluded for evaluation purposes)."

Q: How long does training take? On what hardware?

Answer:

"About 7 minutes for 25 epochs on Apple M-series with MPS acceleration. The bottleneck is the dataloader — negative sampling is single-threaded due to a macOS multiprocessing issue. On a real GPU it'd be ~2 minutes."

5. Loss Functions — The Math

Q: Explain BPR loss.

Math:

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u, i, j)} \log \sigma(\hat{x}_{ui} - \hat{x}_{uj})$$

where $\hat{x}_{ui}$ = score of user u for positive item i , $\hat{x}_{uj}$ = score for negative item j .

Plain English:

"BPR — Bayesian Personalized Ranking — is a pairwise ranking loss. For every (user, positive item, negative item) triplet, it pushes the score of the positive higher than the negative. It uses the sigmoid of the score difference and minimizes negative log-likelihood. The intuition: we don't care about absolute scores, we care that positives rank above negatives."

Q: Explain InfoNCE loss.

Math:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(u, i^+))}{\sum_{j \in \text{batch}} \exp(\text{sim}(u, j))}$$

Plain English:

"InfoNCE — also called sampled softmax — treats retrieval as a classification problem. For each user, the goal is to assign the highest probability to their positive item among all items in the batch. Every other user's positive becomes a negative for free, so with batch size 1024 you get 1023 hard negatives per positive instead of 4. The temperature parameter τ controls how sharp the softmax is — lower τ = harder gradient."

Q: Why did BPR beat InfoNCE on your dataset?

Answer: (This is THE question. Memorize this.)

"I expected InfoNCE to win — it's what Google uses in the YouTube paper, and you get hundreds of negatives per positive. But on ML-1M, BPR beat InfoNCE by 68% on Recall@10.
My hypothesis: ML-1M is heavily popularity-skewed. A few blockbusters like Toy Story and Star Wars dominate user interactions. With InfoNCE in-batch negatives, those blockbusters appear as 'negatives' in almost every batch — including for users who would genuinely love them. The model learns to systematically demote popular items.
BPR with popularity-weighted random sampling gives a softer signal. It still oversamples popular items as negatives (which is correct — they're hard negatives) but doesn't force every batch to contain them.
Lesson: 'use whatever Google uses' is a bad heuristic without checking your data distribution."

Q: How would you fix InfoNCE for skewed data?

Answer:

"Two known techniques from the literature:

1. **Logit correction** (the YouTube paper does this): subtract `log(sampling_probability)` from each logit to correct for the in-batch sampling bias toward popular items.
 2. **Mixed negatives**: combine in-batch negatives with explicitly sampled rare negatives.
- I didn't implement the correction — that's an honest next step."*

Q: Why temperature 0.05?

Answer:

"Lower temperature = sharper softmax = stronger gradient signal. With temperature=1, scores in [-1, 1] give logits with very flat softmax. With temperature=0.05, the same scores spread over [-20, 20], which produces much more decisive gradients. I tried 0.07 and 0.05 — 0.05 worked better for InfoNCE."

6. Inference & Serving

Q: How does inference work?

Answer:

"Three steps for a recommendation request:

1. **Encode user**: pass `user_id` through the user tower → 128-dim vector (~0.15 ms)
2. **FAISS search**: query the pre-built item index for top-K similar items (~0.25 ms)
3. **Filter**: remove items the user already interacted with (lookup in a hash map)

Total end-to-end: ~0.4 ms on Apple Silicon. The item tower is never called online — that's the whole point of the two-tower architecture."

Q: Why FAISS?

Answer:

"FAISS is Meta's library for similarity search — the de-facto standard. I use `IndexFlatIP` which is exact (not approximate) inner-product search. With ~3500 items it runs in 0.25 ms — at this scale, approximate methods like IVF or HNSW add complexity without measurable speedup. For production with millions of items, swapping to `IndexIVFFlat` is a one-line change."

Q: What's the difference between FAISS exact search and approximate search?

Answer:

"Exact (`IndexFlatIP`) compares the query against every item — $O(N)$. Approximate methods like:

- **IVF (Inverted File)**: clusters items, only searches a few clusters → trades a tiny accuracy drop for ~10-100x speedup

- **HNSW (Hierarchical Navigable Small World)**: builds a graph; search walks toward the answer → very fast but more memory
At Google scale (billions of items) you must use approximate. At my scale (3500 items) exact is fine and gives ground-truth recall."

Q: How do you deploy this?

Answer:

"I use FastAPI for the HTTP layer. The model and FAISS index are loaded once at startup. There's a `/recommend` POST endpoint and a `/info` GET endpoint that reports which model variant is loaded. For deployment, I have a Dockerfile that produces a single image. In production at Google scale, this would be a containerized service behind a load balancer, with the FAISS index sharded across replicas."

Q: What's the latency budget? What do you do if it's too slow?

Answer:

"My measured latency is 0.4 ms. Budget at scale would be 10-50 ms p99. If too slow, I'd:

1. Switch FAISS to IVF or HNSW (10-100x speedup)
2. Quantize embeddings (PQ — Product Quantization) — trades a few percent recall for memory & speed
3. Cache user embeddings for active users (hot users get pre-computed embeddings)
4. Use a CPU-friendly model architecture (smaller D, no MLP) for the user tower"

Q: What if a user has never been seen before (cold-start)?

Answer:

"Right now my system rejects unknown users — returns 404. That's a known limitation. In production you'd handle cold-start with:

1. **Content features**: build the user embedding from demographic features (age, country, signup source) instead of an ID lookup
2. **Popular-item fallback**: just return the most popular items
3. **Few-shot bootstrap**: ask the user for 3-5 quick preferences and build a temporary embedding

The two-tower architecture supports content features cleanly — you'd replace `nn.Embedding` with a small encoder over feature vectors."

7. Engineering Practices

Q: How is your code organized?

Answer:

"Modular Python package layout:

- `src/data/` — encoders, dataset, time-based split
- `src/models/` — TwoTowerModel + MFBaseline
- `src/training/` — training loops for InfoNCE and BPR
- `src/evaluation/` — Recall@k, MRR@k, NDCG@k
- `src/indexing/` — FAISS index construction
- `api/` — FastAPI server
- `tests/` — 14 pytest unit tests
- `configs/config.yaml` — all hyperparameters in one place"

Q: How do you test ML code?

Answer:

"Three categories of tests:

1. **Shape & invariant tests:** forward pass produces expected dimensions; embeddings are L2-normalized to within $1e-5$
2. **Behavior tests:** `encode_users` uses the MLP not just the embedding (regression test for the bug I caught); negative sampling never returns positives
3. **End-to-end smoke tests:** metrics return values in $[0, 1]$; FAISS index has the right number of vectors
Total: 14 tests, all passing in CI on every push."

Q: Why CI? What's in the pipeline?

Answer:

"GitHub Actions runs the full pytest suite on every push to main and every PR. Sets up Python 3.11, installs dependencies with caching, runs `pytest tests/ -v`. Catches regressions before they hit main."

Q: Walk me through your Makefile.

Answer:

"One-command workflows:

- `make install` — pip install requirements
- `make data` — download MovieLens-1M
- `make train` — train the model
- `make index` — build FAISS index
- `make compare` — train both losses side-by-side
- `make ablation` — run the full ablation study
- `make test` — run pytest
- `make api-bpr` / `make api-infonce` — serve either variant
- `make docker-build` / `make docker-run` — container workflow"

8. Bugs I Caught

Q: What was the hardest bug you hit?

Answer: (This shows real engineering — memorize the details.)

"The MLP layers were silently never being trained.

The model class defines a 2-layer MLP after the embedding lookup, but my training loop was directly calling `model.user_embedding(...)` and `model.item_embedding(...)` instead of the model's

`forward()` method. So the MLP weights stayed at their random initialization the entire time.

The bug was consistent across training, evaluation, FAISS indexing, and the API — they all bypassed the MLP — so the system worked but as a plain matrix factorization model with a giant unused MLP attached.

How I caught it: I noticed Recall@10 plateaued early. When I checked gradients with

`torch.autograd.grad()`, the MLP gradients were $\sim 1e-9$ — basically zero. Then I traced backwards and found the trainer was bypassing `forward()`.

Fix: Added `encode_users()` and `encode_items()` helper methods on the model that go through the full tower, and updated every caller (trainer, metrics, FAISS, API) to use them. After the fix, Recall@10 jumped because the MLP started actually learning.

I also added a **regression test** so this can't happen again — `test_encode_users_uses_full_tower` asserts that `encode_users` produces different output than raw embedding lookup."

Q: Any other bugs?

Answer:

"Two more, both interesting:

1. OpenMP collision: FAISS and PyTorch each ship their own OpenMP runtime on macOS. Loading both into the same process caused segfaults during InfoNCE's `log_softmax` kernel. Setting `KMP_DUPLICATE_LIB_OK=TRUE` got me past the assertion error, but a deeper issue remained — InfoNCE specifically still segfaulted in `compare.py` even on CPU. I traced it to the order of imports: when `faiss` is imported at module top, it claims OpenMP first, and certain torch operations later collide.

Fix: Lazy-import `faiss` only after training is complete in `compare.py`. That dodged the collision entirely.

2. MFBaseline didn't have `encode_users()`: After my MLP-bug fix, the metrics code called `model.encode_users()` for both the two-tower model and the matrix factorization baseline. The baseline crashed because it didn't have that method. Added matching encoder methods to MFBaseline so the same metric code works for both."

9. Trade-offs & Limitations

Q: What are the limitations of your project?

Be honest. This shows self-awareness:

"Several real limitations:

1. ID-only embeddings: I have no content features. A movie is just an integer to my model. Real systems use genre, cast, embeddings of the title, etc.

2. No sequence modeling: A user is one ID, not a sequence of recent interactions. SOTA systems use

transformers over interaction history.

3. **No cold-start handling**: New users get a 404. Production needs feature-based bootstrap.
4. **No reranker**: Two-tower retrieval is just stage one. Production stacks have a heavier cross-attention reranker on top of the candidates retrieved.
5. **Single dataset**: I only validated on ML-1M. The 'BPR > InfoNCE' finding might flip on other datasets.
6. **No online evaluation**: Offline Recall@10 doesn't always correlate with online business metrics like watch time or session length."

Q: Where would your design break?

Answer:

"Three failure modes:

1. **Skewed user distribution**: My evaluation gives every user equal weight. If 1% of users generate 50% of traffic in production, my Recall@10 isn't measuring what matters.
2. **Distribution shift**: I train on past data and serve on new data. If user behavior changes (new movie release, COVID-style shift), my model doesn't know.
3. **Adversarial inputs**: A malicious user_id like -1 or 99999999 would crash gracefully (404), but I have no rate limiting or auth on the API."

Q: What would you do differently next time?

Answer:

"Five things:

1. **Set up MLflow / W&B from day one** instead of grepping log files for the best metrics
2. **Write tests before code**, especially the regression test for the MLP bug
3. **Run the ablation BEFORE committing to a config** — I built the wrong version first and had to redo it
4. **Use Hydra for configs** — multi-experiment config management is painful with raw YAML
5. **Implement the in-batch logit correction** for InfoNCE so the comparison is properly fair"

10. Scale: How Would Google Deploy This?

Q: How does this scale to YouTube's billion-video catalogue?

Answer:

"Several changes:

Index: Switch from IndexFlatIP (exact) to IndexIVFPQ (inverted file with product quantization). Cuts memory from $O(N \times D)$ to $O(N \times D / 16)$ and search to $O(\sqrt{N})$.

Training: My current setup is single-GPU. At Google scale, you'd use:

- **Distributed data parallel** across many GPUs
- **Sharded embedding tables** (item embedding can be huge — billions of items $\times$ 128 floats = 500GB+)
- **Sampled softmax with logit correction** (the YouTube paper's contribution)

Serving: My API is one Python process. Production needs:

- Horizontal sharding of the FAISS index across replicas

- User embedding cache for hot users
- Multi-tier retrieval (lightweight retrieval → heavyweight reranker)
- Geographic sharding for low latency"

Q: What's the bottleneck if you 100x the data?

Answer:

"Different bottlenecks at different scales:

- **10M interactions**: Negative sampling becomes expensive. Need vectorized batch sampling.
- **100M interactions**: Single GPU memory. Need DistributedDataParallel.
- **1B+ interactions**: Embedding tables exceed single-machine memory. Need parameter server or sharded embeddings (TensorFlow's `EmbeddingVariable`).
- **10B+ items**: FAISS exact search is impossible. Must use approximate. Index might not fit on one machine — need sharding."

Q: How would you A/B test a new version of this model in production?

Answer:

"Standard ML deployment pattern:

1. **Shadow traffic**: route a fraction of requests to the new model but don't return its results — log them and compare offline
2. **Canary**: 1% of real traffic gets the new model, monitor business metrics (CTR, watch time, etc.) for regression
3. **A/B test**: 10% control vs 10% treatment, measure for ~1-2 weeks for statistical significance
4. **Ramp**: 25% → 50% → 100% if metrics are positive

Key gotcha: offline Recall@10 doesn't always predict online win. You can have a model that's 'better' offline but worse online due to filter bubble effects, popularity bias, etc."

11. Math You Must Know Cold

Recall@k

$$\text{Recall@k} = \frac{1}{|U|} \sum_{u \in U} \mathbb{1}[\text{any relevant item in top-k}]$$

"Fraction of users for whom at least one of their actual test items appears in our top-k recommendations."

MRR@k (Mean Reciprocal Rank)

$$\text{MRR@k} = \frac{1}{|U|} \sum_{u \in U} \frac{1}{\text{rank of first relevant item}}$$

"Reward shows up earlier in the list — rank 1 = 1.0, rank 2 = 0.5, rank 3 = 0.33, etc. Capped at k."

NDCG@k (Normalized Discounted Cumulative Gain)

$$\text{DCG@k} = \sum_{i=1}^k \frac{\text{rel}(i)}{\log_2(i+1)}$$

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

"Like MRR but rewards multiple relevant items at different positions. The log discount means rank 1 is way more valuable than rank 10."

Cosine Similarity (= Dot Product on Unit Vectors)

$$\cos(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}$$

"Measures angle, not magnitude. After L2 normalization, denominator is 1, so it's just the dot product."

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

"Squashes any real number to (0, 1). Used in BPR for the score-difference probability."

12. Behavioral Questions

Q: Tell me about a time you had to debug something hard.

Answer: Use the MLP bug story (Section 8). Structure as STAR:

- **Situation:** Training loss was decreasing but Recall@10 plateaued early
- **Task:** Figure out why the model wasn't improving
- **Action:** Checked gradients → found MLP grads were ~0 → traced to trainer bypassing forward()
- **Result:** Fixed it, added a regression test, Recall@10 improved significantly

Q: Tell me about a time you disagreed with conventional wisdom.

Answer: Use the BPR-vs-InfoNCE finding.

- **Situation:** YouTube paper says InfoNCE; everyone uses it for two-tower
- **Task:** Validate that on my dataset
- **Action:** Ran controlled comparison; BPR won by 68%
- **Result:** Investigated why (popularity skew), wrote it up, defended the non-conventional choice

Q: What's something you learned the hard way?

Answer:

"Random splits leak the future into your training data. My first model had Recall@10 of 0.72, which felt amazing — until I realized I was leaking. Time-based split dropped it to 0.47, which is honest. Now I always default to time-based splits unless there's a specific reason not to."

Q: How do you decide what to work on next?

Answer:

"I prioritize by signal-per-hour. For this project, after the basic model worked, the highest-leverage moves were: (1) ablation study to validate config choices, (2) the loss comparison because it produced the most interesting finding, (3) tests because regressions are expensive. Polishing the README came last because it's only valuable once the underlying work is solid."

13. Curveball Questions

Q: Your Recall@10 is 0.57. That sounds low — most movies aren't in the top 10. How do you defend that number?

Answer:

"Two points:

- 1. **It's not 'top 10 of all movies'** — it's **top 10 of 3,468 candidates per user**. Random would be ~0.3% (10/3468). My number is 165x random.*
- 2. **Compare against real baselines, not intuition**: published two-tower results on ML-1M typically hit 0.30-0.55 Recall@10. LightGCN (a graph neural net) gets ~0.27. My 0.57 is competitive with state-of-the-art on this dataset.*

What feels low intuitively is high in absolute terms once you remember the candidate set size."

Q: You used Claude / AI to help build this. Why should I believe you understand it?

Answer:

"Fair question. Three answers:

- 1. **I can whiteboard any line right now**. Pick a function, I'll explain it.*
- 2. **The bug-finding is mine** — caught the MLP bypass, the OpenMP issue, the data leakage. AI doesn't catch these in your specific code.*
- 3. **The interpretive work is mine** — the BPR-beats-InfoNCE finding required running the experiment, looking at the result, and forming a hypothesis about why. That's the part AI can't fake.*
I used AI like I'd use Stack Overflow or a senior engineer — for boilerplate, debugging, syntax. The architecture, design decisions, and analysis are mine."

Q: I see your CI is passing. What if I push a change that breaks the model output but not the tests?

Answer:

"That's a real gap. My tests cover shapes, invariants, and basic behaviors but don't catch model-quality regressions. To close that, I'd add:

1. **Quality regression test:** train a tiny model on a fixed seed, assert Recall@10 > some threshold
2. **Snapshot test:** feed a fixed user_id, assert top-3 items match a golden output
3. **Property-based test:** assert that more training improves recall (with hypothesis library)"

Q: Is your project original work?

Answer:

"The architecture is from the literature — that's the point. I didn't invent two-tower retrieval; I built and validated an implementation of it. The original contributions are:

1. The empirical comparison of BPR vs InfoNCE with the popularity-skew explanation
2. The full ablation study with controlled experiments
3. The production engineering (Docker, CI, tests, latency benchmarking)

I'm not claiming a research paper. I'm claiming I can take a paper, build it correctly, validate it rigorously, and ship it."

14. Live Demo Script

If they ask you to demo, follow this exact flow:

Step 1: Show the README

"Let me start with the architecture diagram..." (point to mermaid diagram)

Step 2: Show the test suite passing

```
make test
```

"14 tests covering models, data, metrics, and indexing. CI runs this on every push."

Step 3: Run the model

```
make api-bpr
```

"Serving the BPR variant. Now let me hit the API:"

```
curl -X POST http://localhost:8000/recommend \  
-H "Content-Type: application/json" \  
-d '{"user_id": 1, "top_k": 10}'
```

"~0.5ms latency, top-10 movies for user 1."

Step 4: Show the comparison

"Here's the side-by-side comparison of InfoNCE vs BPR..." (open `comparison_results.md`)

"BPR won by 68%, here's my hypothesis why..." (give the popularity-skew explanation)

Step 5: Show the ablation

"And here's the full ablation table proving the MLP matters and embedding dim is saturated..."

Step 6: Show the bug story

"Speaking of which — let me tell you about the bug I caught..." (open `DECISIONS.md`)

Final Pre-Interview Checklist

The night before:

- ☐ Read this entire document
 - ☐ Practice the 30-second pitch out loud 5 times
 - ☐ Run `make test` — make sure it still passes
 - ☐ Run `make serve` — make sure latency still ~0.4 ms
 - ☐ Open one random file from `src/` and explain it to yourself
 - ☐ Re-read `DECISIONS.md`
 - ☐ Make sure GitHub repo is public and CI badge is green
 - ☐ Have the URL ready to paste into the chat
-

What If You Don't Know an Answer?

Don't bullshit. Say:

"I haven't thought about that. My instinct is X, but I'd want to verify. Can I think out loud?"

Then reason through it. Interviewers respect honesty + reasoning over confident wrong answers.

You got this. The work is real. Defend it like the work it is.